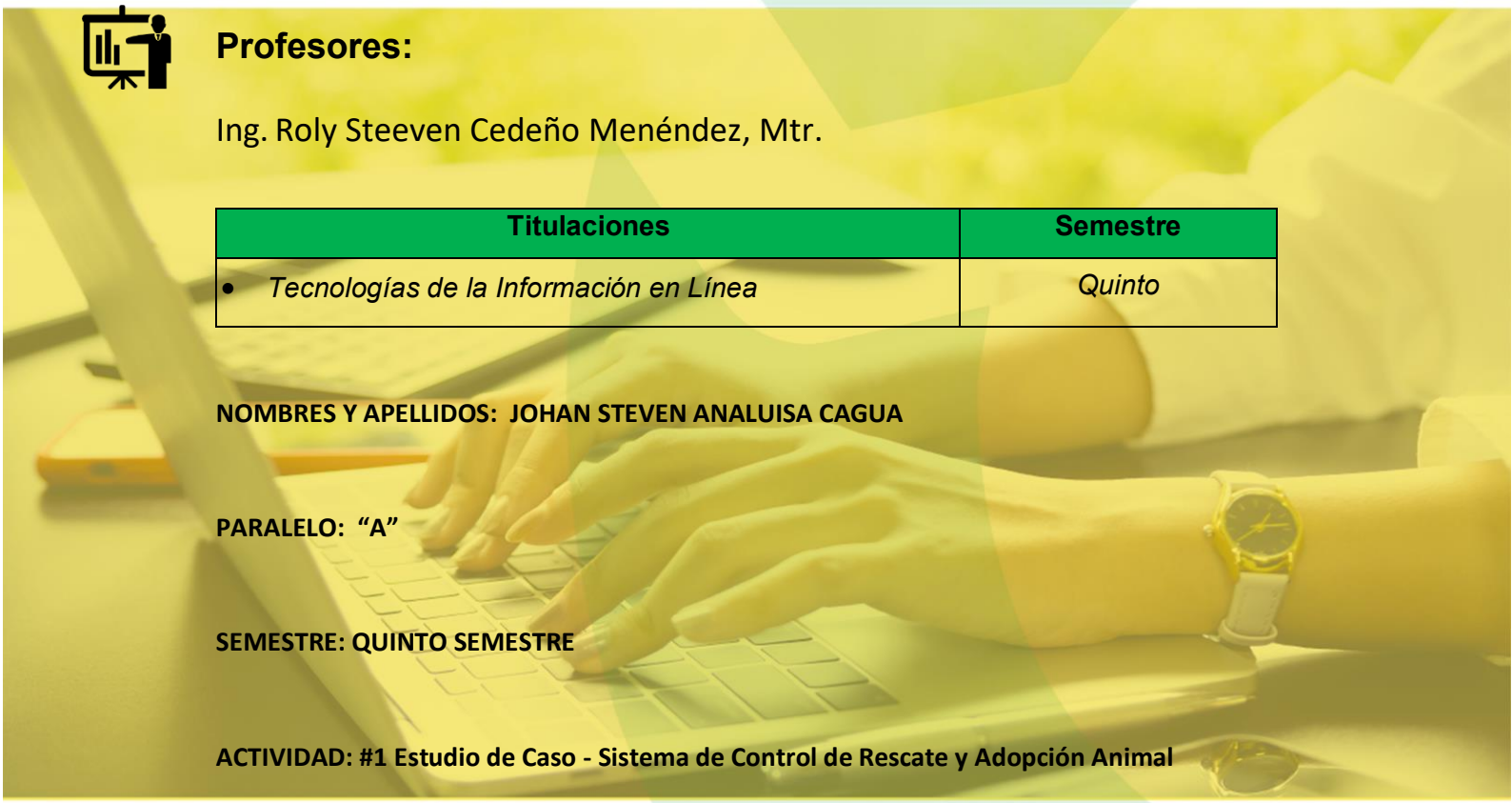
TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: JOHAN STEVEN ANALUISA CAGUA

PARALELO: "A"

SEMESTRE: QUINTO SEMESTRE

ACTIVIDAD: #1 Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2025 - ENERO 2026

1. ANÁLISIS DEL PROBLEMA

El sistema de Control de Rescate y Adopción Animal tiene como propósito facilitar la gestión de los casos de animales rescatados dentro de un refugio animal. El sistema permitirá registrar los ingresos de animales rescatados, clasificarlos según su especie, asignar responsables para su atención y realizar el seguimiento de su estado durante el proceso de rescate, tratamiento y adopción.(avaq, 2023)

La solución estará orientada a mejorar la organización y trazabilidad de los casos gestionados por el refugio, permitiendo que los diferentes usuarios involucrados puedan realizar las actividades correspondientes de acuerdo con sus responsabilidades.

El sistema deberá contemplar animales de diferentes categorías, principalmente perros, gatos y fauna silvestre, y permitirá gestionar el proceso desde el momento en que se registra un rescate hasta las etapas posteriores de atención y adopción.

1.1 Actores principales

De acuerdo con el escenario planteado, se identifican los siguientes actores:

Ciudadano / Reportante: Es la persona que reporta un animal que requiere rescate. Su participación se relaciona principalmente con el ingreso de información necesaria para registrar un nuevo caso de rescate.

Voluntario / Veterinario: Es el encargado de atender o dar seguimiento a los animales rescatados. Puede recibir casos asignados y participar en el proceso de atención del animal, incluyendo su tratamiento y seguimiento.

Administrador del Refugio: Es el responsable de administrar los casos dentro del sistema. Entre sus funciones se encuentra gestionar los registros de rescate, categorizar los animales, asignar voluntarios o veterinarios y realizar el seguimiento de los casos.

1.2 Problema identificado: La gestión de animales rescatados requiere organizar adecuadamente la información de cada caso, identificar la especie del animal, asignar responsables y mantener actualizado el estado del proceso. Por esta razón, se plantea el desarrollo de un sistema que centralice la información de los casos de rescate y permita realizar un seguimiento organizado de cada animal, facilitando la coordinación entre ciudadanos o reportantes, voluntarios, veterinarios y administradores del refugio.(wikipedia, 2022)

1.3 Objetivo del sistema

Diseñar una arquitectura lógica para un sistema de gestión de rescate y adopción animal que permita registrar y administrar casos de animales rescatados, categorizar las especies, asignar responsables y controlar el estado de cada caso, aplicando buenas prácticas de diseño de software, patrones de diseño y un flujo profesional de control de versiones mediante Gitflow.(FUNIBER, 2005)

2. REQUERIMIENTOS DEL SISTEMA

Los requerimientos del sistema permiten definir las funcionalidades que deberá proporcionar el Sistema de Control de Rescate y Adopción Animal, así como las características de calidad que deberá cumplir para garantizar una solución escalable, mantenible y organizada.(TRBL, 2023)

2.1 Requerimientos funcionales

Los requerimientos funcionales describen las acciones que el sistema deberá permitir realizar a los usuarios.

Crear caso de rescate: El sistema deberá permitir registrar un nuevo caso de rescate de un animal, almacenando la información necesaria para iniciar el seguimiento del caso.

Actor relacionado: Ciudadano/Reportante y Administrador del Refugio.

Registrar información del animal

El sistema deberá permitir registrar la información correspondiente al animal rescatado asociado a un caso.

Actor relacionado: Administrador del Refugio.

Categorizar la especie

El sistema deberá permitir clasificar al animal rescatado de acuerdo con su especie. Las categorías consideradas en el escenario son:

- Perro.
- Gato.
- Fauna silvestre.

Actor relacionado: Administrador del Refugio.

Asignar voluntario o veterinario

El sistema deberá permitir asignar un voluntario o veterinario responsable de atender y realizar el seguimiento de un caso de rescate.

Actor relacionado: Administrador del Refugio.

Gestionar el estado del caso

El sistema deberá permitir realizar el seguimiento del estado del animal durante el proceso de atención. Entre los estados contemplados se encuentran:

- Rescatado.
- En Tratamiento.

- Adopción.

Actor relacionado: Voluntario/Veterinario y Administrador del Refugio.

Consultar información de los casos

El sistema deberá permitir consultar la información registrada de los casos de rescate para facilitar su seguimiento y gestión.

Actor relacionado: Voluntario/Veterinario y Administrador del Refugio.

2.2 Requerimientos no funcionales

Los requerimientos no funcionales establecen las características de calidad que deberá cumplir el sistema.

Escalabilidad: El sistema deberá estar diseñado de manera que pueda incorporar un mayor número de casos de rescate, animales, usuarios y responsables sin requerir modificaciones importantes en su arquitectura.

Mantenibilidad: El sistema deberá utilizar una estructura organizada y modular que facilite la incorporación de nuevas funcionalidades, corrección de errores y mantenimiento del código.

Trazabilidad de versiones: El desarrollo del sistema deberá utilizar un sistema de control de versiones que permita registrar y consultar los cambios realizados durante el desarrollo del proyecto.

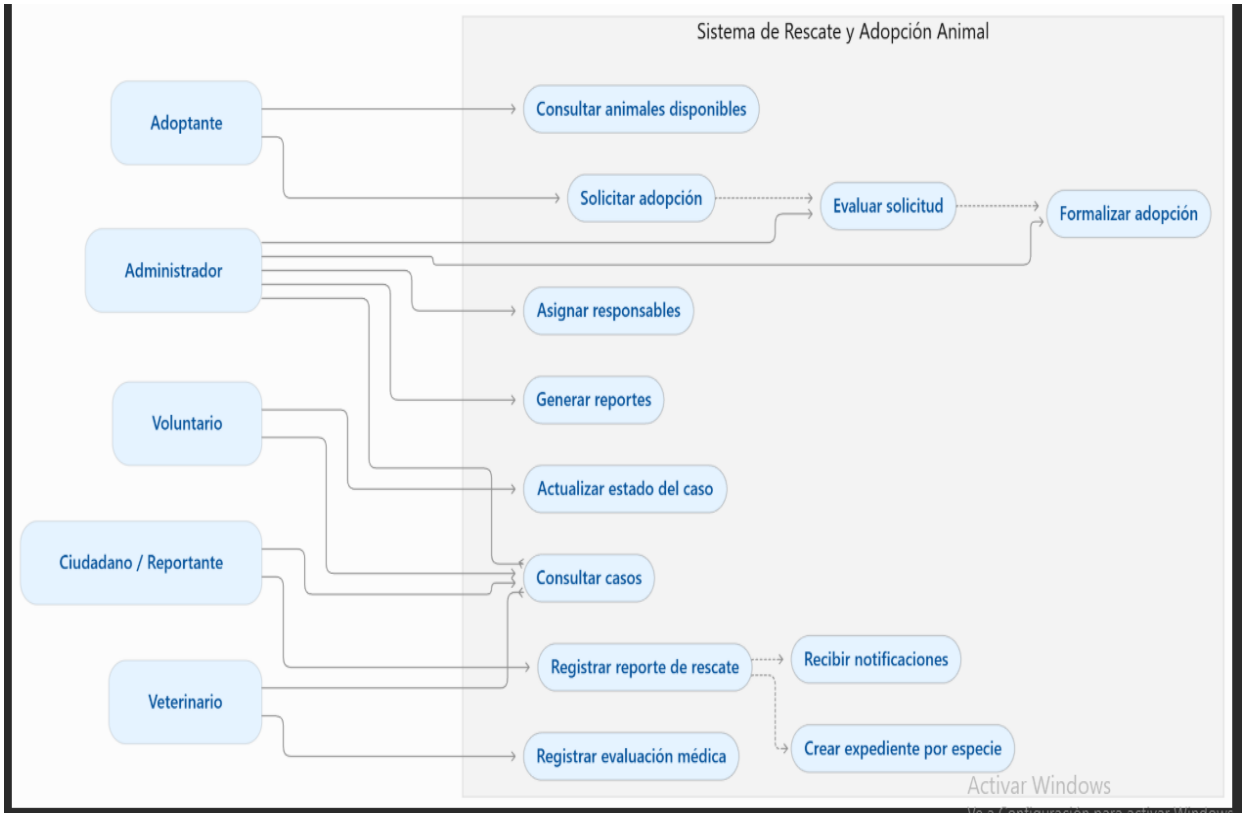
Para este propósito se utilizará un repositorio público en GitHub siguiendo el flujo de trabajo Gitflow.

Reutilización: El diseño deberá favorecer la reutilización de componentes y soluciones mediante la aplicación de patrones de diseño de software.

Claridad y organización: La arquitectura y el código deberán mantener una estructura clara, utilizando nombres significativos para las clases y métodos y aplicando principios de buenas prácticas de ingeniería de software.

Diagrama UML de casos de uso

Este diagrama muestra las funciones del sistema y las interacciones de los usuarios con la plataforma.

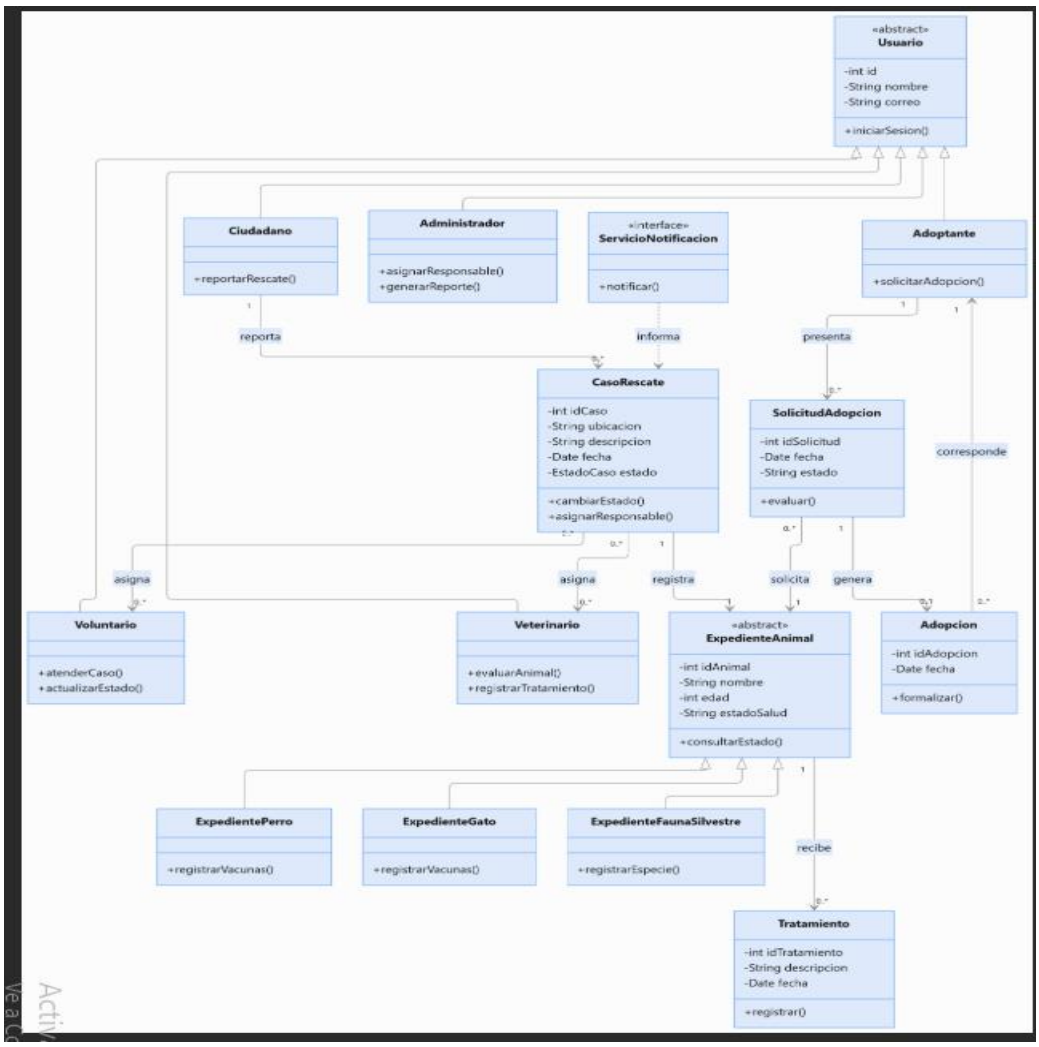


Explicación: el ciudadano registra un caso de rescate; el administrador asigna a los responsables; el voluntario y el veterinario actualizan la información de atención. Cuando un animal está disponible para adopción, el adoptante puede presentar una solicitud que será evaluada antes de formalizar el proceso.

En UML formal, las relaciones include y extend deben utilizarse con su significado específico. Las líneas discontinuas del esquema anterior representan dependencias conceptuales; al elaborar la versión definitiva en StarUML o Draw.io, conviene etiquetarlas correctamente o utilizar asociaciones simples.

Diagrama UML de clases

El modelo de clases define los principales objetos del sistema y sus relaciones. Se propone una estructura orientada a objetos que facilita aplicar principios SOLID, separar responsabilidades y extender los tipos de animales.


«abstract»
Usuario
-int id
-String nombre
-String correo
+iniciarSesion()
Ciudadano
+reportarRescate()
Administrador
+asignarResponsable()
+generarReporte()
«interface»
ServicioNotificacion
+notificar()
Adoptante
+solicitarAdopcion()
reporta
informa
presenta
CasoRescate
-int idCaso
-String ubicacion
-String descripcion
-Date fecha
-EstadoCaso estado
+cambiarEstado()
+asignarResponsable()
SolicitudAdopcion
-int idSolicitud
-Date fecha
-String estado
+evaluar()
corresponde
asigna
asigna
registra
solicita
genera
Voluntario
+atenderCaso()
+actualizarEstado()
Veterinario
+evaluarAnimal()
+registrarTratamiento()
«abstract»
ExpedienteAnimal
-int idAnimal
-String nombre
-int edad
-String estadoSalud
+consultarEstado()
Adopcion
-int idAdopcion
-Date fecha
+formalizar()
ExpedientePerro
+registrarVacunas()
ExpedienteGato
+registrarVacunas()
ExpedienteFaunaSilvestre
+registrarEspecie()
recibe
Tratamiento
-int idTratamiento
-String descripcion
-Date fecha
+registrar()

Implementación conceptual de tres patrones de diseño

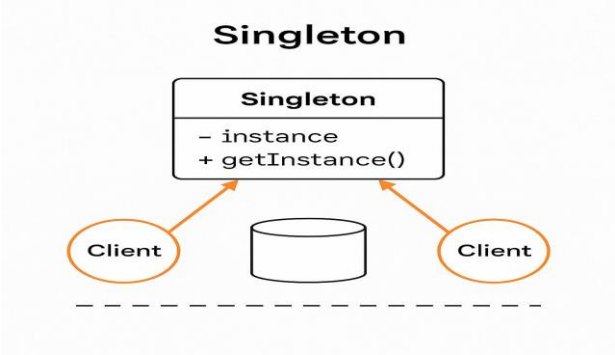
La guía recomienda explícitamente Singleton, Observer, Factory Method y MVC. Para cumplir el requisito de tres patrones, propongo implementar los tres primeros y utilizar MVC como organización general de la aplicación.

Singleton — Creacional

Problema: se necesita un punto central para gestionar las operaciones de los casos.

Solución: una clase GestorCasos mantiene una única instancia compartida dentro de la aplicación.

Beneficio: centraliza el acceso al servicio y evita crear múltiples instancias del gestor de casos por error.

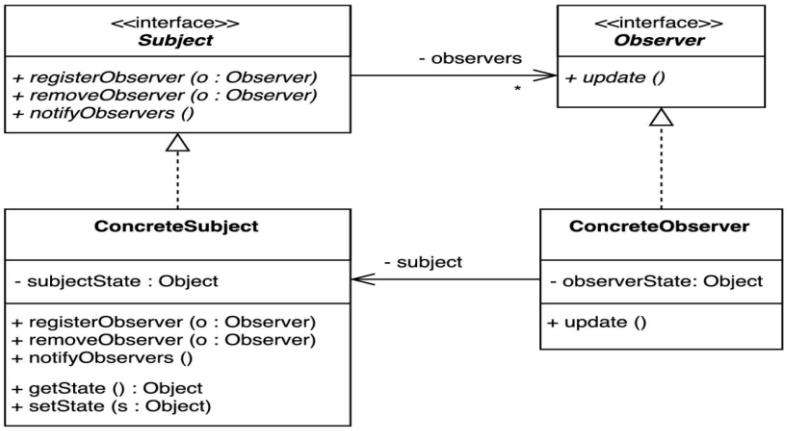


Observer /Comportamiento

Problema: al registrar un rescate, los responsables deben recibir una notificación sin que el registro dependa de un canal concreto.

Solución: el caso publica un evento y los observadores suscritos reciben la notificación.

Beneficio: permite agregar nuevos destinatarios o canales sin modificar la lógica principal del rescate.

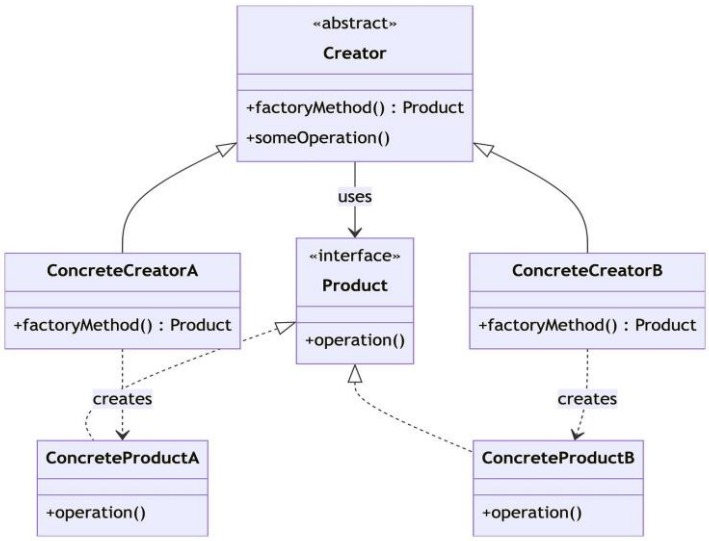


Factory Method/Creacional

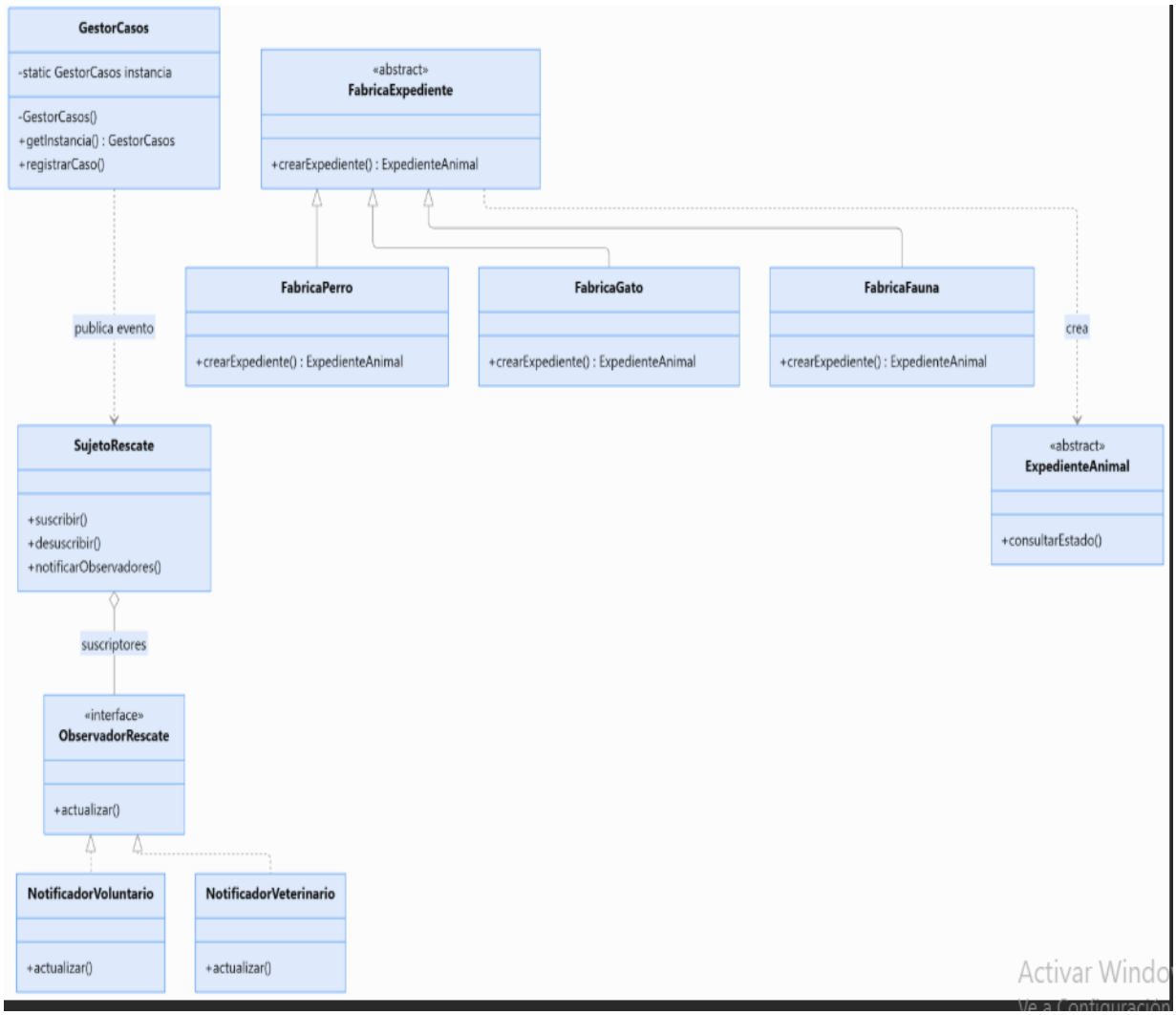
Problema: la creación de expedientes varía según la especie del animal. Solución: una fábrica define la operación de creación y las fábricas concretas producen expedientes para perros, gatos o fauna silvestre.

Beneficio: evita dispersar condicionales por todo el sistema y facilita incorporar nuevas especies.

.



Ejemplo de clases de los patrones

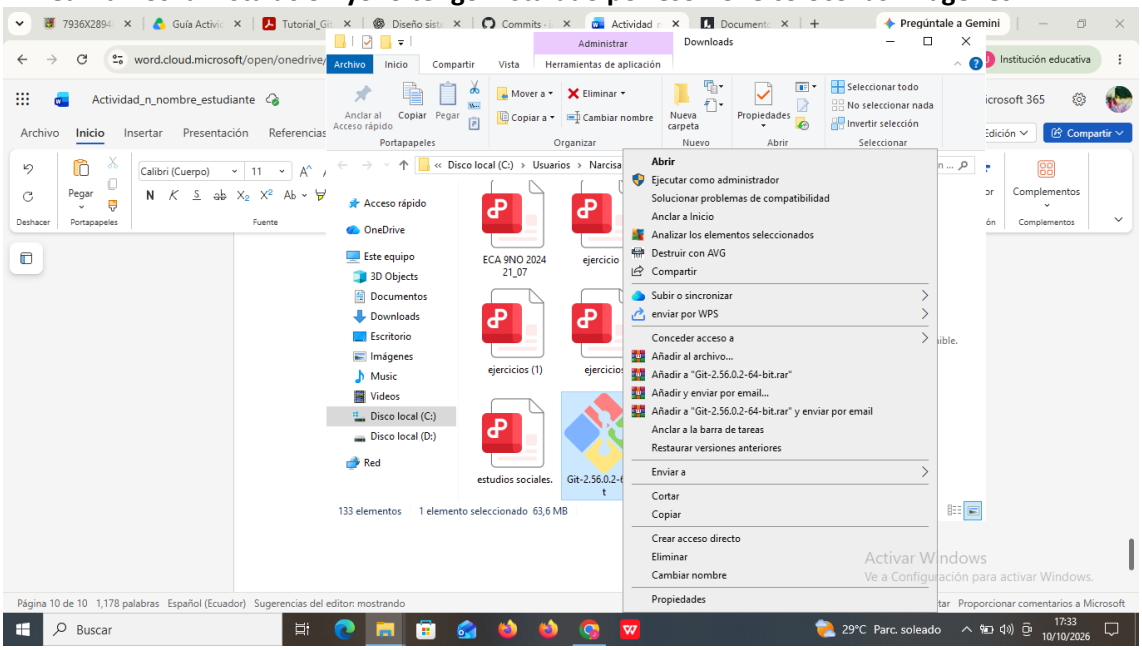


Crear el repositorio:

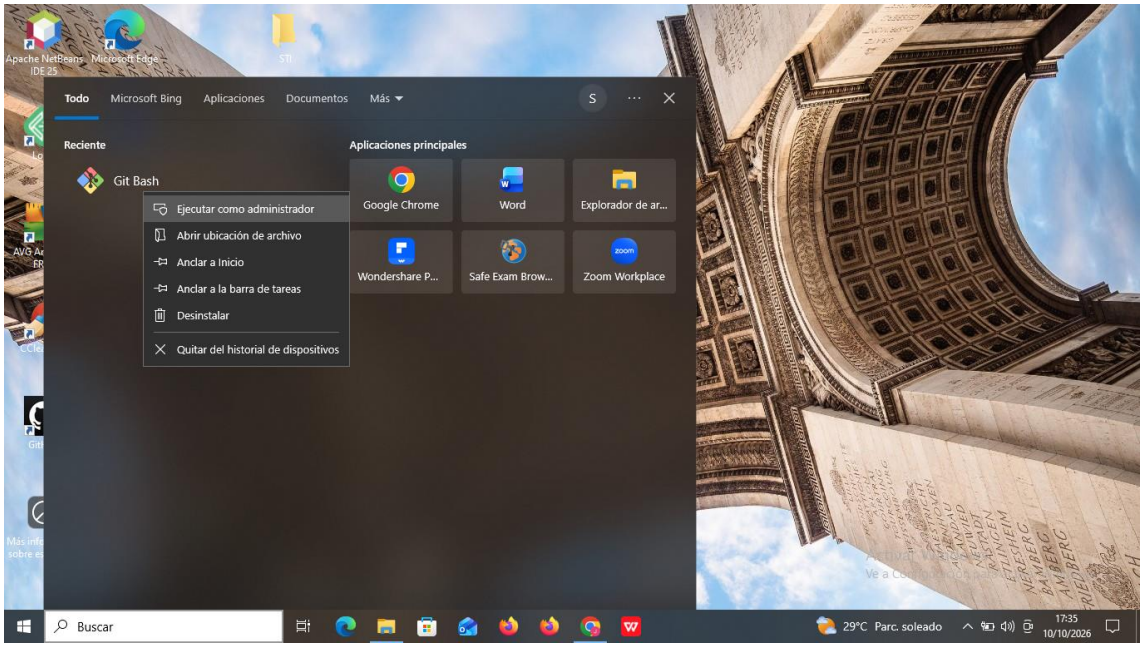
1. Buscamos y descargamos el aplicativo para windows

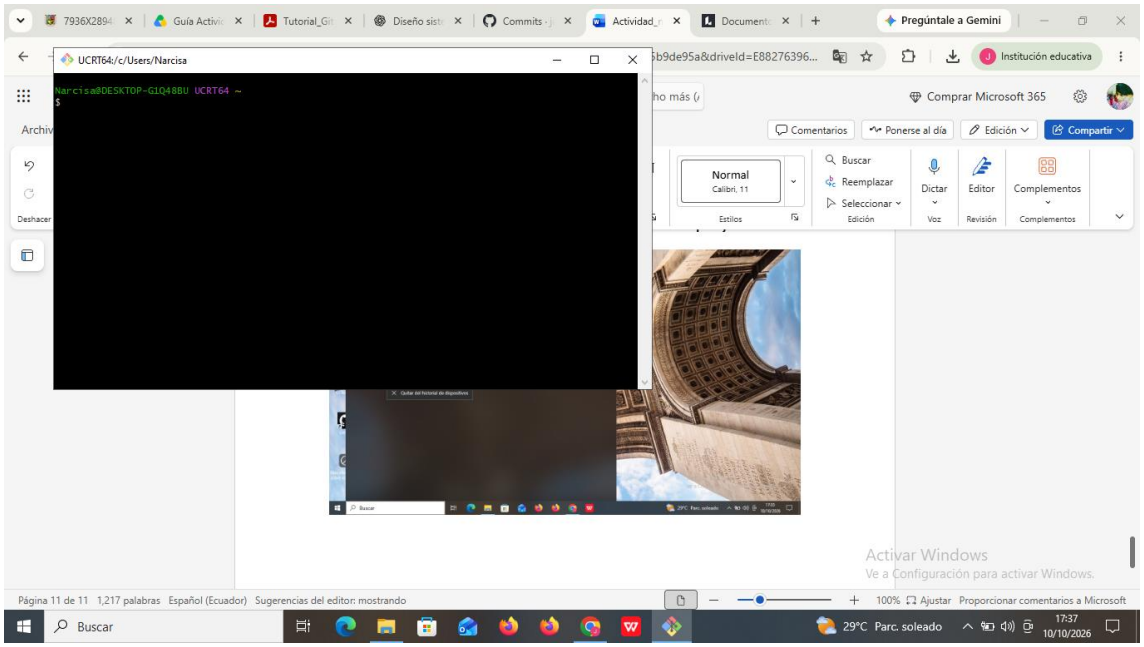


2. Realizamos la instalación yo lo tengo instalado por eso no le coloco las imagenes.

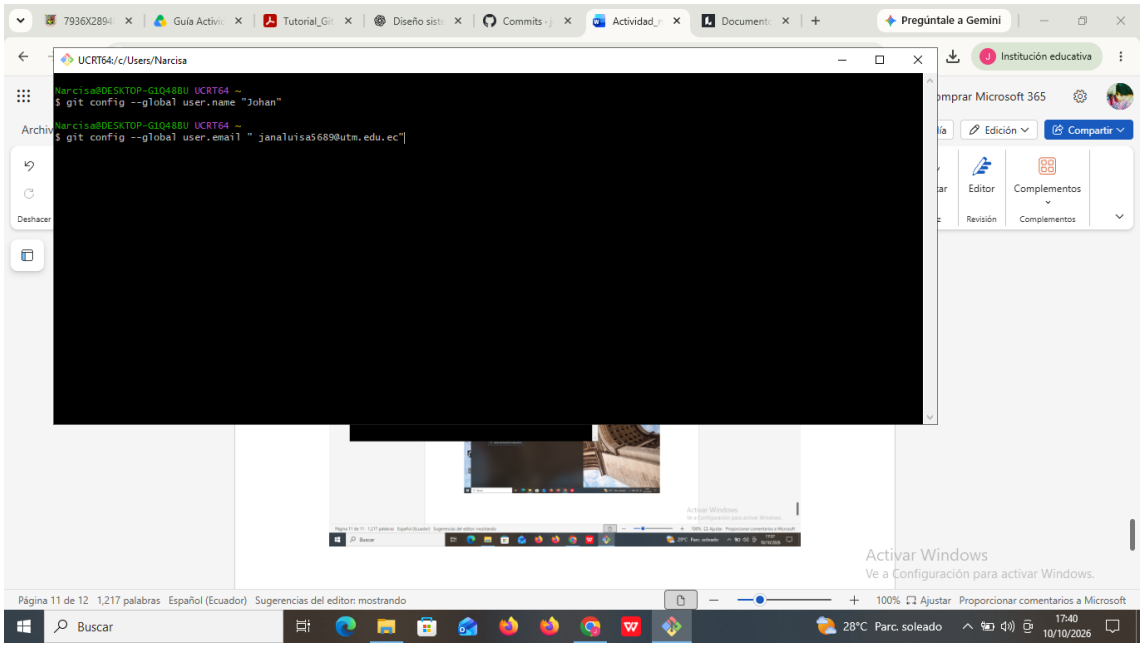


3. Una vez instalado se procede abrir la aplicación un dato yo tuve que ejecutarla como administrador para poder guardar el nombre y correo.

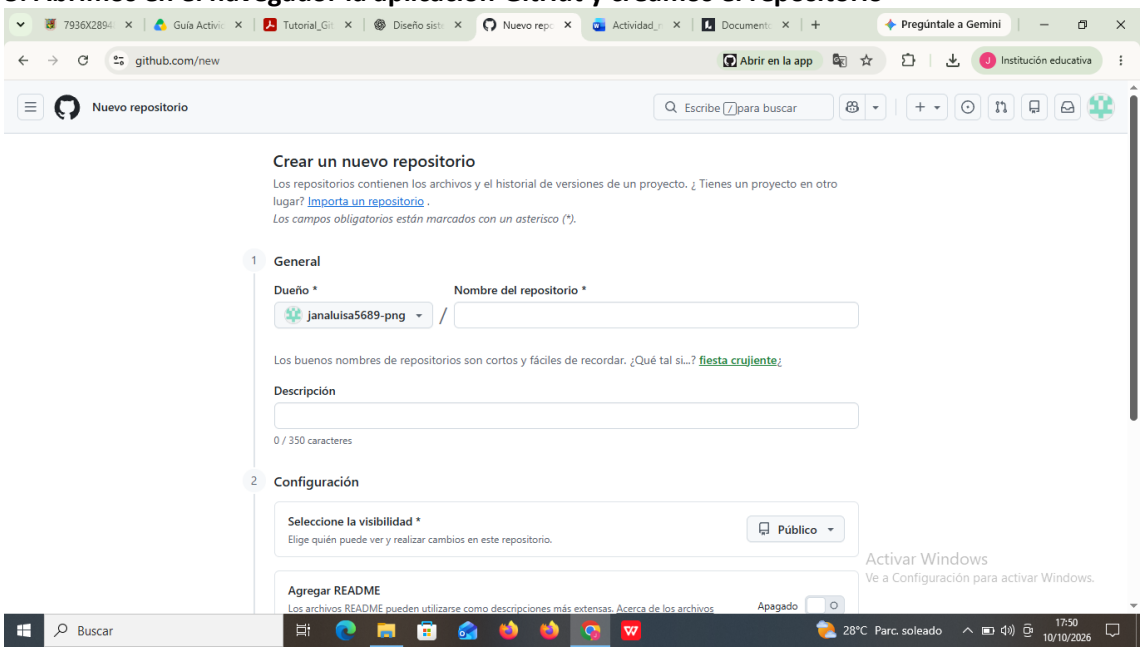




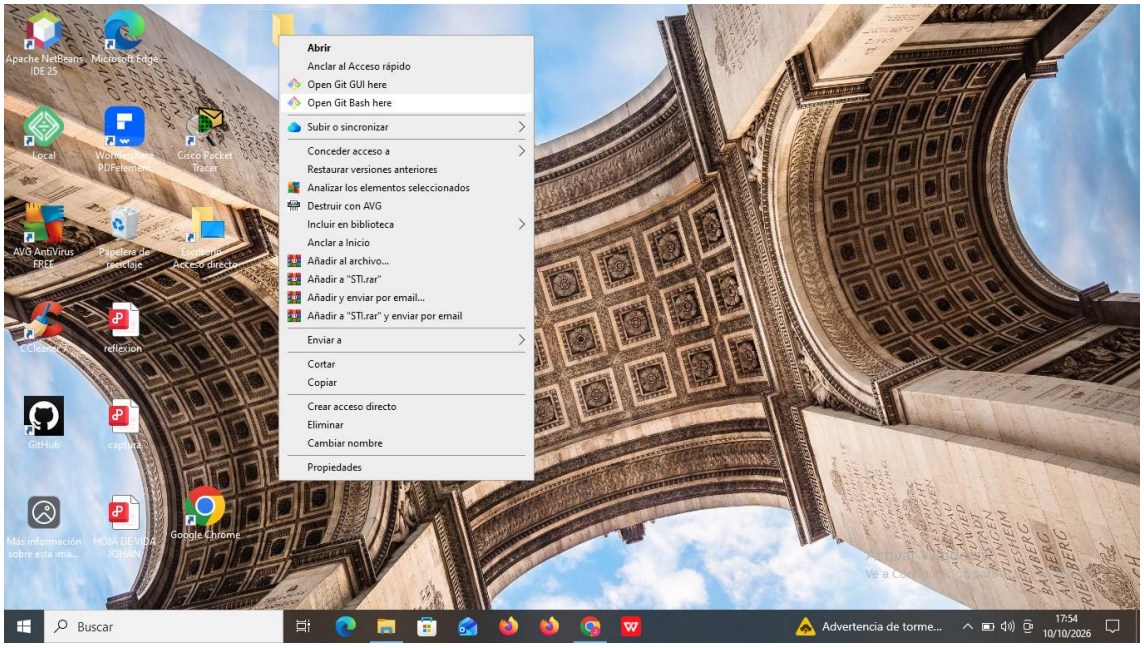
4. Ejecutamos estos códigos para que se registre nuestra información



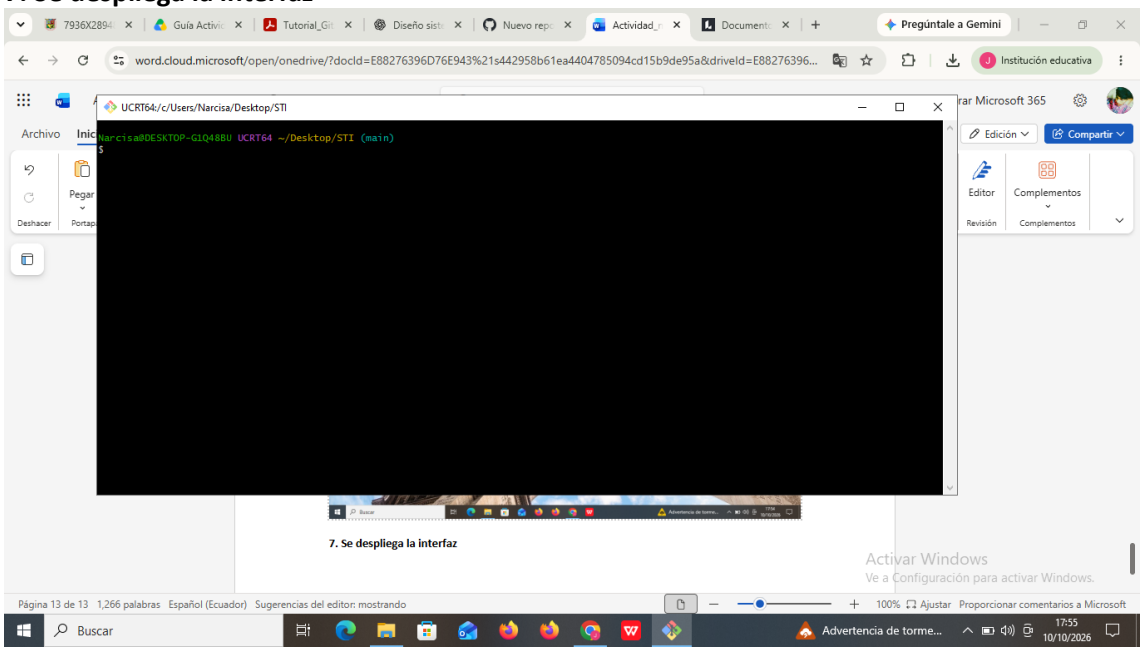
5. Abrimos en el navegador la aplicación GitHut y creamos el repositorio



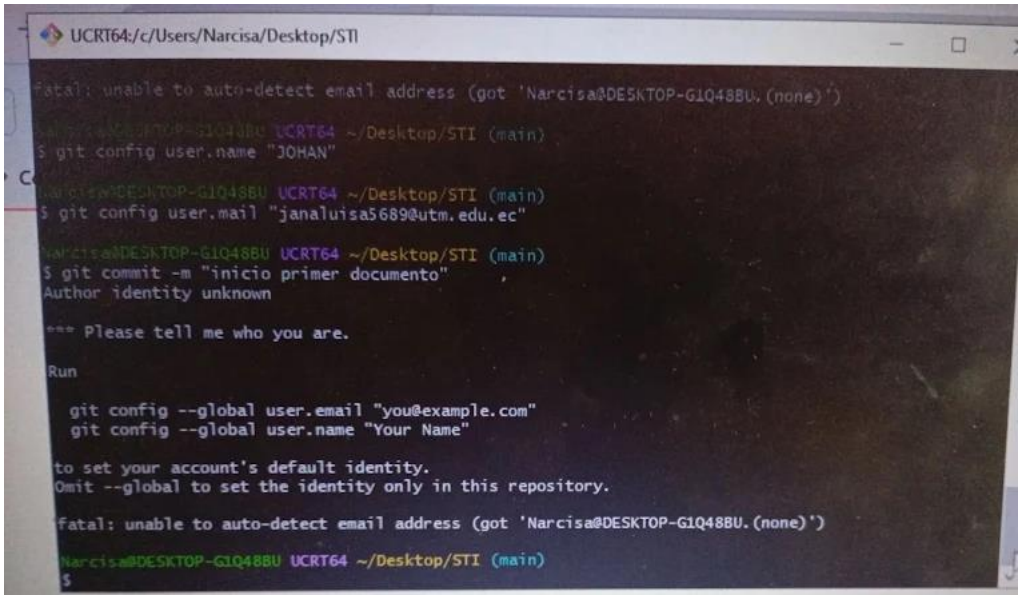
6. Creamos una carpeta en el escritorio donde vamos a trabajar con el proyecto, le damos clic derecho y le damos a:



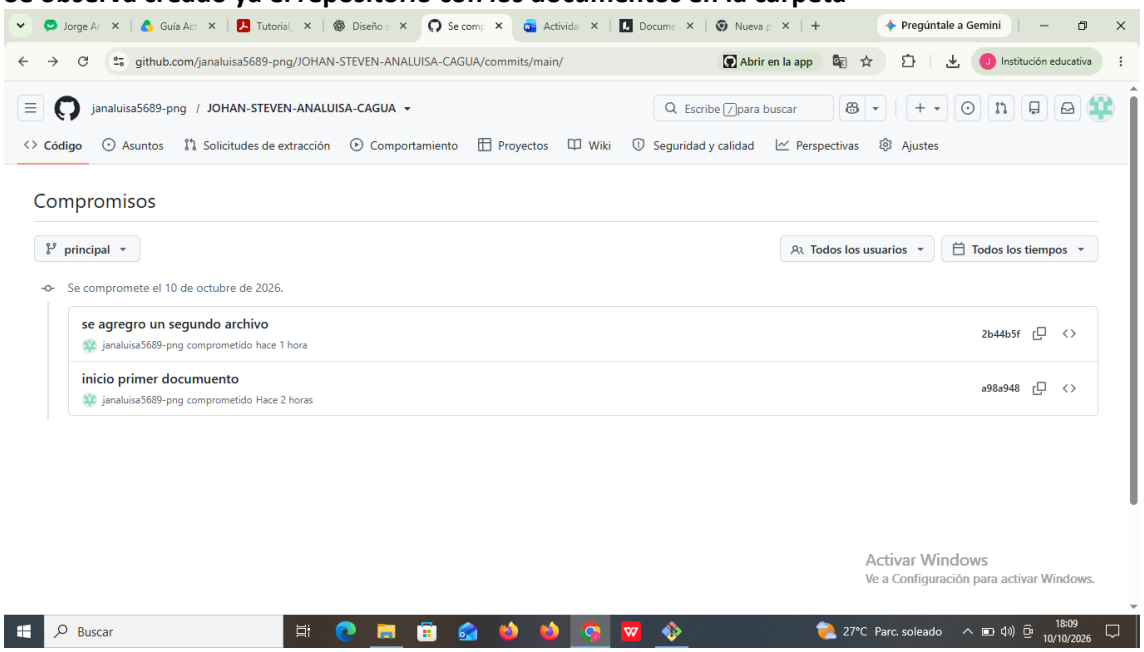
7. Se despliega la interfaz



Ejecutamos los codigo para iniciar el nuevo repositorio
Git init
Luego se coloca este codigo para poder realizar cambios
git add .
Git commit –m “primer Documento”



Se observa creado ya el repositorio con los documentos en la carpeta



LINK DE LA CARPETA DEL REPOCITORIO:

https://github.com/janaluisa5689-png/JOHAN-STEVEN-ANALUISA-CAGUA/commits/main/

BIBLIOGRAFIA

avaq. (2023). *¿Qué es un dispositivo lógico programable complejo (CPLD)?* https://lucid.app/lucidchart/8f07e3cc-4fc7-4489-812d-

239a9945c4fb/edit?beaconFlowId=FF083DB8E8C024A9&invitationId=inv_f1d34259-bbc0-422a-b9f5-c9df3f628f48&page=0_0#

FUNIBER. (2005). *Estratégica en Ingeniería de Software*. Estratégica En Ingeniería de Software.

TRBL. (2023, May 12). *Sistema embebido,Tecnologías* . Https://Trbl-Services.Eu/Blog-Los-Sistemas-Embebidos-La-Clave-Internet-Las-Cosas/.

wikipedia. (2022, August 23). *Circuito integrado de aplicación específica*. https://es.wikipedia.org/wiki/Circuito_integrado_de_aplicaci%C3%B3n_espec%C3%ADfica